

Job Sequencing and Tool Switching

What limits an exact method, and where the limit comes from

Shankar

LIMOS — Université Clermont Auvergne

Advisors: Prof. Rafael Colares, Prof. Annegret Wagler

Academic supervisor: Dr. Renaud Chicoisne

2 September 2026

One quantity runs the whole problem

One line of arithmetic — the tools some job needs, minus what fits in the magazine —

$$q = |\mathcal{U}| - b$$

decides *both* how good the standard industrial heuristic is, *and* whether any exact method can close an instance.

1. The heuristic

The textbook worst case is not a worst case. The gap is a price, and the price can be named.

2. The exact methods

Three unrelated formulations all stop at q . Measured, not conjectured.

3. What to do about it

Why every inequality we have is too local, and a family that is not.

Where this goes

Part I — the heuristic

1. The textbook worst case has no gap
2. The gap is the price of fewest groups
3. An open question of Prof. Wagler's, closed

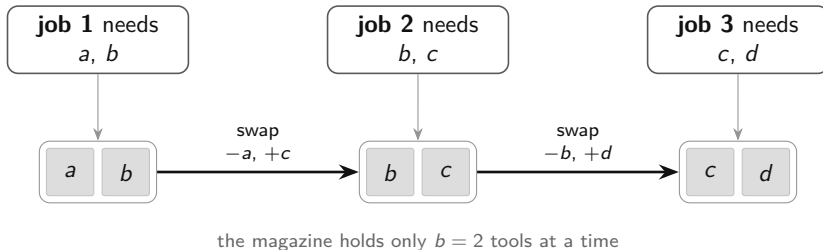
Part II — the exact methods

4. One bound decides every outcome
5. Why that bound will not move
6. A family of inequalities that could

Everything measured here comes from 16,994 solver runs and is re-derivable from the raw files by one command.

The problem

One machine, many tools, a magazine that holds only a few



- Each stop costs production time, and a cell running thousands of parts a day repeats it constantly. Same structure in circuit assembly, colour printing, picking.
- **Choose the order of the jobs and the loading at each step** so that the total number of tool insertions is smallest.

Two decisions that behave completely differently

Loading is easy.

For a *fixed* order, the cheapest policy is known in closed form: *Keep Tool Needed Soonest* (Tang & Denardo, 1988). When a tool must go into a full magazine, remove the one whose next use is furthest away.

Optimal, and $O(n|T|)$.

Ordering is hard.

NP-hard for every fixed capacity $b \geq 2$ (Crama et al., 1994).

A good order puts jobs with similar requirements next to each other — but “similar” means only *overlapping sets*, with no natural distance, and the magazine limit couples jobs that sit far apart in the order.

An easy inner problem inside a hard outer one — exactly what decomposition is built for.

How the loading works, on four jobs

Magazine holds $b = 3$; grey = tool sits in the magazine

	job 1 a, b	job 2 b, c	job 3 c, d	job 4 d, a
tool a	+			
tool b	+			
tool c		+		
tool d			+	

tool a is kept instead —
job 4 will want it

tool b is dropped here:
it is never needed again

Four insertions in total. The only decision is what to drop when the magazine is full, and the rule is *drop the tool whose next use is furthest away*. That rule is provably optimal once the job order is fixed — so this half of the problem has been solved since 1988.

Nobody solves it directly

The two-phase method, and the question nobody had answered

1. **Group** the jobs into as few magazine-fitting batches as possible.
2. **Sequence** the batches.
3. **Re-optimize the loading** across the whole flattened order.

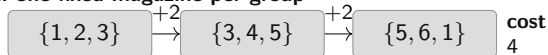
It is fast, it is what the strongest metaheuristics start from, and it is what industry uses.

Known: a b -approximation (Crama & van de Klundert, 1999).

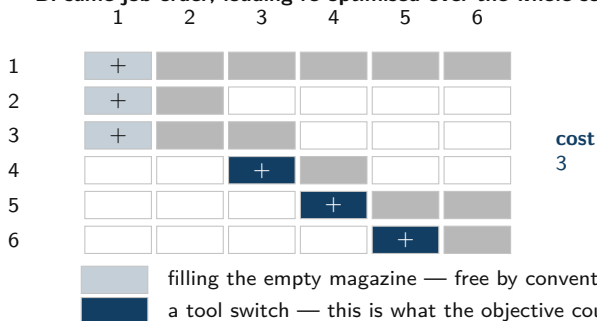
How far from optimal can it be? On which instances is it exact?

Result 1. The same grouping, measured two different ways

A. one fixed magazine per group



B. same job order, loading re-optimised over the whole sequence



The literature reports **A**; the method as specified does **B**. Every ring from $k = 3$ to 9 has gap zero under B.

Result 1. The textbook worst case is not a worst case

The literature's canonical bad example is the k -ring. Its reported gap is measured with *one fixed magazine per group* — step 3 of the method is left out.

k	K^*	Z^*	H_{walk}	walk gap	heuristic gap
4	2	1	1	0	0
6	3	3	4	1	0
7	3	4	5	1	0
8	3	5	6	1	0
9	4	6	7	1	0

Every k -ring, $k = 3, \dots, 9$: the method as actually specified is **optimal**.

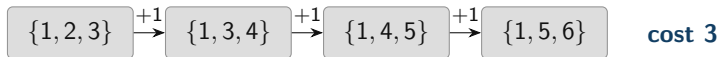
The published gaps are walk values. Separating the two readings is what makes the rest of the analysis correct.

Result 2. The gap is exactly the price of using the fewest groups

Fewest groups — what the heuristic is told to do



Best over *all* groupings — what the optimum uses



Theorem. $Z^* = \min_{\mathcal{P}} \gamma(\mathcal{P})$ over *all* feasible groupings, of every cardinality.

So the gap is not a failure of the sequencing phase — that orders whatever it is given optimally. It is the price of the cardinality restriction, and nothing else.

What follows from it

- **Zero-gap classes for every capacity** — families on which the two-phase method is provably exact.
- **Worst-case guarantees** better than the trivial $b(K^* - 1)$; extremal behaviour settled exactly at three groups.
- **A census.** In 1,260 random instances at three to five jobs, *none* has a positive heuristic gap; extending it finds 3 of 1,260 at six jobs and 2 at seven. Every positive gap found anywhere equals *one*. Capacity is what admits them: none at all in 11,000 draws at $b = 2$, but they occur at $b = 3$ — the smallest has five jobs.
- **An open question of Prof. Wagler's, closed.** At $b = 3$ a job is an *edge* and a tool a *vertex*, so the family is finite and can be enumerated. Odd rings are *not* the only obstruction to integrality — the smallest is a three-job **star**, which has no odd cycle at all.

The census is exhaustive on small instances. Burger et al. report from a *printing press* that the decomposition degrades as the magazine grows relative to the largest job. **Two completely different methods, the same regime.**

Result 3. An open question, closed

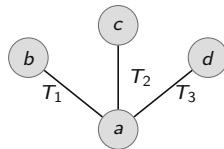
Grouping branch-and-price is fast only when its covering relaxation is ideal

Odd rings are one obstruction. **Are they the only one?**

At $b = 3$ with two tools per job the question becomes *finite*:

a tool is a **vertex**, a job is an **edge**,
a feasible group is a subgraph on ≤ 3
vertices,
and the k -ring is the cycle C_k .

So the whole family can be enumerated: **151**
instances up to isomorphism.



any two jobs fit in a magazine of three;
all three need four tools

No. 76 of 151 are non-integral, and **eleven are bipartite** — no odd cycle at all. The smallest obstruction is this **three-job star**: fractional cover $\frac{3}{2}$ against $K^* = 2$. The odd ring needs five jobs.

The exact side, and the wall it runs into

Every compact formulation proposed in forty years has a weak linear relaxation. The strongest of them relaxes to

$$q = |\mathcal{U}| - b$$

— every tool that does not fit in the initial magazine must be inserted at some point. The strongest relaxation available is this counting argument.

Two methods built to get past it:

- **Branch-and-Benders-cut** — put the ordering in a master problem and settle the loading exactly with the polynomial oracle, so no weak relaxation is ever formed.
- **Position-indexed branch-and-price** — PCF' prices one magazine per position; PTF prices *pairs* of consecutive magazines, and its relaxation can provably exceed q .

Result 3. The bound decides the outcome

1,421 *instances* · 12 *configurations* · 16,994 *runs* · *one hour each*

	instances	Benders	SSPMF
coverage bound tight ($Z^* = q$)	523	94%	100%
coverage bound loose ($Z^* > q$)	592	40%	87%

- Same solver, same hardware, same time limit. The classification is not by number of jobs, tools, or density — it is by one structural property.
- Of the runs that hit the limit and whose optimum is known, 66% **already held that optimum** and could not certify it.
- Across 41,673 pairs of methods that both proved an optimum, **none disagreed**.

The shape of the failure, not just its size

	instances closed within a given time					
	1s	10s	60s	300s	900s	3600s
SSPMF	482	735	946	959	982	1038
LSS	57	246	532	759	819	844
BBC-LP	403	469	525	572	650	726

- The Benders solver closes **403 instances in the first second** and 726 in the full hour. More than half of everything it will ever solve is done before a compact model has finished building. Then it flattens.
- The compact models start lower and keep climbing. **More time would not help the two equally.**
- Where it does not finish: median gap at the limit **22.9%**, and only 50 of 681 runs get within five per cent of proven.

It is sitting on the right answer — optimal 59% of the time — with a quarter-sized hole in the certificate.

The same number, from a completely different direction

PCF' root relaxation equals q	2,114 of 2,114 runs
PTF root relaxation exceeds q	3 of 233 runs

- PCF': **proved** in the report, then confirmed on every single run, across six different acceleration settings. Not one exception.
- PTF: the proposition is **not vacuous** — but the advantage is one unit, on three small instances, against gaps of six to eight.
- Benders master, PCF', and the literature's multicommodity model: three unrelated routes, one number.

This is a property of the problem, not of any one implementation.

Result 4. Why the bound will not move

Write β_t for the number of blocks of positions in which tool t sits in the magazine:

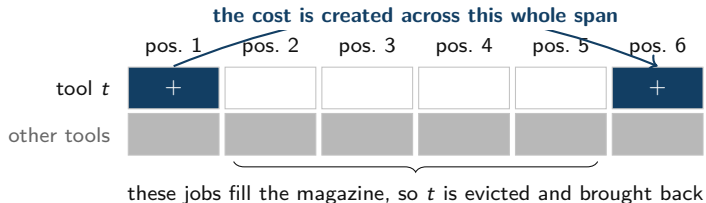
$$Z^* = \sum_{t \in \mathcal{U}} \beta_t, \quad \beta_t \geq 1 \implies Z^* \geq q.$$

Everything above q is a **re-insertion** — a tool evicted and brought back. That happens because of the *global interleaving* of the schedule.

But every inequality either method has is *local*.

- Benders: one aggregated row over *arcs*; triplet rows that are worth zero at a fractional point; and every optimality cut is arc-supported too.
- PCF': one configuration per *position*, coupled only per tool — so the relaxation mixes fractionally and pays no transition.
- **Measured:** on 87.4% of Benders runs the root LP equals $|\mathcal{U}|$ exactly. The pairwise and triplet rows contribute *nothing*.

Why a re-insertion is invisible to the inequalities we have



What the cost really is

$Z^* = \sum_t (\text{blocks of tool } t).$

One block each gives exactly q .

Everything above q is a **re-insertion**, caused by what sits *between* two uses of a tool.

What our inequalities see

The Benders master indexes everything by **arcs** — pairs of neighbours. PCF' indexes by a **single position**.

Neither can see a span. Measured: on 87.4% of runs the master's root bound is exactly q and nothing else contributes.

Result 5. A family that is not local, and what it is worth

For a stretch of positions $W = [p, r]$: every tool needed inside W is either already loaded at $p - 1$, or inserted within W .

$$\sum_t \sum_{k \in W} w_{t,k} \geq |\{t \text{ present somewhere in } W\}| - \sum_t a_{t,p-1}$$

Valid — checked at 1,200,816 integer points, no violation. And **robust**: only master variables, so the pricing oracle is untouched.

Best bound each family could *ever* certify, on the 62 loose instances:

family	mean	median	no gain on
arc-supported — what both solvers carry today	8.9%	0%	46/62
window	30.9%	40%	18/62

The arc family is *exhausted*. The window family has headroom at its *ceiling* — but see the next slide: the ceiling did not transfer.

And then we measured it

- The window family was added to the PCF' master — 15 windows, 660 rows, 225 variables per instance — and the root relaxation recomputed on 37 loose instances.
- **The bound moved on none of them.** Root = q with the rows and without, including instances whose gap is ten or eleven.
- **Why.** Every window row is *slack* at the LP optimum. The relaxation keeps each tool partly present at every position, so $\sum_t \max_{k \in W} a_{t,k}$ stays near b and no tool ever *enters* a window.

A ceiling computed over *integral* sequences is not a bound on a *fractional* relaxation. That is the lesson, and it is in the report.

Four things that did not work — and why that is the evidence

- **Fractional Benders cuts.** 67.5 million generated; node count fell twentyfold; the bound at termination rose on *no* instance; 67 solved instances lost. They are arc-supported, so they re-derive what the integer cuts already say.
- **Pareto-optimal cut lifting.** −37 instances. Choosing a deeper cut inside the wrong family.
- **Conflict-graph row.** Neutral — as predicted, the coverage row dominates it.
- **Only the primal strengthening helped (+31):** a hybrid genetic warm start. A solver whose difficulty was finding solutions would not behave this way.
- **Learned cut selection.** Built and trained, but on knapsack cover cuts, *not* on the SSP. It learns — 46–73% over random selection — and says nothing about this problem, which is why the report puts it in an appendix. Selection cannot pick a strong cut that does not exist.

These are not four separate disappointments. They are *four independent measurements of the same thing*: every one of them attacks the bound, and every one of them fails in the way the diagnosis predicts.

Conclusions

- The two-phase heuristic admits **two readings**, and the literature's worst case belongs to the wrong one. The method is better than the model of it.
- The gap is exactly the **price of using the fewest groups** — with zero-gap classes, worst-case bounds, and a census that agrees with the one industrial study.
- A configuration-based exact method for the SSP is **bound-limited, not implementation-limited**. Established four independent ways.
- The bound will not move because every inequality we have is **local** and the cost is not. A non-local family exists and is valid — but in its robust form it leaves the root bound unchanged.

Next: a non-robust encoding of the window family, which costs the pricing oracle; chain PTF per configuration rather than per tool; disaggregate the Benders master's single θ . And the open problem — *is the excess of PTF over q bounded by a constant?*

Thank you

Report, code, benchmark data and the independent verification program:

`github.com/shankar-n/JGP-SSP-Codes-Shankar`

Every number in this talk is reproduced from the raw result files by

`verification/analyse_campaign_results.py --check`

Backup — instances solved, all twelve configurations

Configuration	Catanzaro	Crama	Laporte3	Laporte4	Laporte5	Laporte7	All
SSPMF	73/171	68/160	340/340	330/330	161/336	66/80	1038/1417
CATZ-F4	55/171	44/160	340/340	330/330	72/327	44/78	885/1406
LSS	52/132	42/124	340/340	330/330	39/101	41/42	844/1069
BBC-LP+T	38/168	34/156	340/340	192/330	84/336	39/80	727/1410
BBC-LP	38/169	35/151	340/340	191/329	83/338	39/80	726/1407
BBC-K	39/166	34/145	340/340	190/329	83/335	39/80	725/1395
BBC-LP+F+H	43/171	37/160	316/340	157/330	97/339	40/80	690/1420
BBC-LP+F	37/169	32/160	313/340	156/329	85/340	36/80	659/1418
BBC-LP+F+C	37/170	32/160	311/340	157/329	84/336	36/79	657/1414
BBC-LP+ACC	43/170	37/160	286/340	154/330	97/339	39/80	656/1419
BBC-K+F	37/171	21/121	316/340	156/329	86/338	35/79	651/1378
BBC-LP+F+P	35/171	32/160	282/340	155/328	83/336	35/79	622/1414

Denominators are runs that returned a result. 427 of 16,994 failed; 352 are LSS. Those losses are not random — SSPMF proves 78% of what LSS completed and 60% of what it lost — so LSS never attempted the hard third and its row is not comparable. The three compact models are not cleanly separable; all three beat every configuration built here — and CATZ-F4 is Catanzaro's F4, not the F5 they recommend, so that margin is if anything understated.

Backup — the ablation

Configuration	what it adds	solved	difference
BBC-LP+F	—	659	—
BBC-LP+F+H	hybrid-genetic incumbent	690	+31
BBC-LP+F+C	conflict-graph row	657	−2
BBC-LP+ACC	all three together	656	−3
BBC-LP+F+P	Pareto-optimal cut lifting	622	−37
BBC-LP	<i>fractional separation removed</i>	726	+67

Removing a feature outperforms every addition.

Only the strengthening that improves the *incumbent* helps. Every attempt at the *bound* costs more than it returns.

Backup — the two cost conventions

Formulations disagree on whether the initial magazine is free. The two costs differ by the size of the initial load, so native objectives are **not** comparable across methods.

- Every comparison in this work is on the *empty-start* cost of the returned sequence, re-evaluated by the loading oracle.
- SSPMF reports the free-initial cost. Difference = b on 1,029 of its 1,038 closed instances; smaller on the other nine — exactly those whose optimal sequence starts with a job needing fewer than b tools.
- An instance must be keyed by *(family, name, jobs, tools, capacity)*. Crama reuses file names across four capacities; keying without it manufactures disagreements that do not exist.
- Ten instances declare a tool that no job requires, so the bound is $|\mathcal{U}| - b$ and not $|T| - b$.

Backup — learned cut selection, on knapsack cover cuts

knapsack size n	learned policy	random selection	improvement
10	1.3855	0.8734	+58.6%
12	1.1639	0.7014	+65.9%
20	0.7482	0.5021	+49.0%
22	0.6692	0.4568	+46.5%
24	0.6383	0.3888	+64.2%
26	0.5661	0.3279	+72.7%

Mean tightening of the linear bound on held-out instances. One training seed per size, 2,000 episodes.

The machinery learns. This is not the SSP, and no claim about the SSP is made from it.

Backup — the window inequality

For a stretch of positions $W = [p, r]$, every tool needed inside W is either already loaded at $p - 1$ — where the magazine holds exactly b — or inserted within W :

$$\sum_t \sum_{k \in W} w_{t,k} \geq |\{t \text{ present somewhere in } W\}| - \sum_t a_{t,p-1}$$

- **Proved**, not merely tested. One auxiliary variable per tool and window linearises it.
- **Robust**: only master variables appear, so its dual never reaches the pricing problem. The set-union knapsack oracle is untouched.
- The per-tool counting rows already in the master are the case $W = [2, n]$, so this *generalises* what is there.
- 84% of the windows that carry the bound have length ≤ 4 , so a static pool suffices — no separation routine needed.

Backup — how to check any number in this talk

- `verification/verify_report_independent.py` — re-derives every small-instance claim from scratch, sharing no code with the solvers. **57 checks, all passing.** No commercial solver needed.
- `verification/analyse_campaign_results.py --check` — recomputes every figure in the computational section from the raw per-instance files and compares it against what the report states.
- `verification/bound_probe.py` — reproduces the ceiling comparison between the two inequality families.
- `src/BNP/window_cuts.py` — the implementation, with a brute-force validity self-test over 1,200,816 integer points.

The loading rule is checked against an exact dynamic program on 349,872 instance-and-sequence pairs, and across 41,673 pairs of methods that both proved an optimum, none disagree.